

Sparsity-Adaptive Gated Eviction for Memory-Efficient Long-Context Inference of Large Language Models

First Author¹ Second Author¹ Third Author²

¹ [Department, University, City, Postcode, Country]

² [Department, University, City, Postcode, Country]

first.author@example.edu third.author@example.edu

Abstract We introduce SAGE, a training-free framework that compresses the key-value cache of large language models during autoregressive inference through confidence-gated eviction. Existing eviction methods apply a uniform per-layer budget and rely on static importance scores, so quality degrades sharply once the cache is reduced below half of its full size. SAGE couples three mechanisms: an exponentially decayed attention-confidence score that tracks the current relevance of every cached entry, a layer-adaptive budget allocator that distributes cache capacity across layers according to measured attention saturation, and a dual-reservoir structure that protects a recent window while evicting only from the history region. Experiments on LLaMA-2-7B/13B-Chat and Vicuna-13B show that SAGE holds WikiText-2 perplexity within 0.35 points of the full cache at a 20% cache budget (5.46 against 5.12 on the 7B model), retains 98.0% of the LongBench average, and reaches 94.8% needle-retrieval accuracy at a 128K context, where the strongest eviction baseline TOVA obtains 6.53, 94.9% retention, and 81.6%. At 32K tokens SAGE shrinks the cache from 16.8 GB to 3.8 GB and raises decode throughput by 1.52x, the speedup growing to 2.02x at 128K, which extends the context capacity of a 24 GB GPU from 17K to 74K tokens.

Keywords Large language models · KV cache compression · Long-context inference · Attention sparsity · Memory efficiency

1 Introduction

Autoregressive decoding in decoder-only large language models requires a key-value (KV) entry for every token generated so far, because each attention layer re-reads the full history when producing the next position [1, 2]. This design scales from few-shot learners trained at web scale [3] through the open families of LLaMA, OPT, and Mistral [4–7], and the footprint of the cache grows linearly with the context: a 7B-parameter model in half precision spends about 16.8 GB on the cache at 32K tokens and more than 67 GB at 128K, and the cache, rather than compute, becomes the binding constraint of long-context deployment [8, 9]. Two families of methods reduce the footprint. Quantization lowers the bytes per entry [10–12], while eviction drops entries judged unimportant [13–16]. Eviction is attractive because it needs no calibration data and no change to the kernel layout, yet current methods lose quality quickly once the cache falls below roughly half of its full size.

Our preliminary study of uniform-budget eviction methods exposes three failure modes. First, attention maps are far more concentrated in deep layers than in shallow ones, so a uniform per-layer budget wastes capacity where attention already saturates and starves the layers that still need wide support. Second, accumulated-attention scores never decay, which lets a token that received heavy attention early in a document hold its slot long after the discussion has moved on. Third, pure importance-based eviction can sacrifice the most recent tokens, and the resulting loss of local attention structure shows up as a collapse of retrieval accuracy on long contexts. Each failure mode is visible in isolation, which suggests that a single mechanism cannot fix all three at once.

We propose Sparsity-Adaptive Gated Eviction (SAGE), a training-free framework that addresses the three failure modes with three coupled components under one global memory contract. An attention-

confidence scoring rule (ACS) accumulates the attention each entry receives and decays it exponentially, so stale importance yields to current relevance. A layer-adaptive budget allocator (LABA) measures attention saturation online, expressed as the normalized entropy of the layer’s averaged attention distribution, and shifts capacity toward the layers whose attention is still diffuse. A dual-reservoir eviction policy (DRE) splits the cache into a protected recent window plus confidence-selected history and evicts only from the history region, while attention sinks [15] remain permanently resident. The three components share one scoring array, and the complete metadata, the score array plus the bucket structure, costs 2.1% of the cache bytes it controls, and the realized footprint including the clipping floor and bookkeeping is 22.6% of the full cache against the nominal 20% contract. SAGE requires no model updates, no calibration corpus, and no architecture change, so it applies to any decoder-only model with a standard attention implementation.

This paper makes the following contributions.

- We introduce SAGE, a KV cache eviction framework that unifies decayed confidence scoring, layer-adaptive budget allocation, and a dual-reservoir cache structure under a single global memory contract.
- We isolate the contribution of each component in a one-at-a-time ablation study: layer-adaptive budgets cut the WikiText-2 perplexity degradation of a uniform-budget variant from 0.69 to 0.34 at a 20% budget, and replacing static with decayed scoring removes a further 0.28 points of degradation in its own ablation, while the protected recent window prevents the retrieval collapse that pure importance eviction suffers.
- We evaluate SAGE on three models, two perplexity corpora, the eleven LongBench tasks, and needle retrieval up to 128K tokens. Under a 20% budget SAGE retains 98.0% of the LongBench average and 94.8% retrieval accuracy at 128K, compresses the KV memory 4.4x at 32K tokens, and speeds decoding up to 2.02x at 128K, extending the context capacity of a 24 GB GPU from 17K to 74K tokens.

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 formalizes budget-constrained eviction. Section 4 presents the SAGE design. Section 5 reports the experiments. Section 6 discusses the findings, Section 7 states the limitations, and Section 8 concludes the paper. Code, configurations, and the full experimental record needed to reproduce every table and figure will be released with the paper.

2 Related work

2.1 KV cache compression by eviction

One line of research identifies droppable entries through attention statistics. H2O shows that accumulated attention concentrates on a small set of heavy hitters and keeps a recent window alongside the heavy hitters [13]. ScissorHands formulates a persistence-of-importance hypothesis and selects tokens whose importance persists across the text stream [14]. TOVA evicts greedily per attention head, keeping the entries that the current query assigns the largest mass, with a cache size that adapts to the attention distribution rather than a preset budget [16]. FastGen profiles attention patterns per head and applies pattern-specific compression strategies [17]. PyramidKV observes that pyramid-shaped per-layer budgets follow the information funnel of deep networks [18]. These methods share one design choice that SAGE revisits: the per-layer budget is uniform in most implementations, and the importance score is

static or slowly varying. SAGE differs on both points. The budget varies across layers according to a measured saturation signal and is re-allocated online, and the confidence score decays explicitly, which we show in Section 5.6 is what separates near-lossless compression from sharp degradation at aggressive budgets.

A parallel line compresses bytes per entry rather than the number of entries. LLM.int8() quantizes activations at scale [10], KIVI stores keys and values in 2-bit asymmetric quantization [11], and KVQuant pushes million-token contexts with non-uniform, outlier-aware quantization [12]. Eviction and quantization act on orthogonal dimensions, the entry count against the bytes per entry, so the two can be stacked. This paper concentrates on the eviction dimension and reports the combination with quantized caches as future work.

2.2 Sparse and efficient attention

Training-side sparse attention restricts the key set that each query can see. Longformer and Big Bird impose fixed sparse patterns [19, 20], the Sparse Transformer introduces factorized block patterns [21], Adaptively Sparse Transformers learn per-head attention spans [22], ALiBi extrapolates positions to lengths unseen in training [23], and Transformer-XL recycles segment-level states [24]. These methods require training or fine-tuning, which makes them complementary to inference-time compression rather than substitutes. Weight pruning is a third, again orthogonal, family that removes parameters rather than cache entries [25]. On the inference side, StreamingLLM discovers attention sinks and keeps a local window, which enables streaming generation [15]. FlashAttention reduces the IO cost of exact attention without changing its semantics [26, 27], and systems such as vLLM manage cache pages to raise serving throughput [9]. SAGE belongs to the inference-side family, but it takes the memory contract, not the window shape, as the control variable, and it drives eviction with a global decayed confidence signal rather than query-local scores.

2.3 Long-context evaluation

Evaluation of long-context ability spans three levels. Perplexity on WikiText-2 [28] and PG19 [29] measures language quality under compression. LongBench aggregates multilingual long-context tasks that exercise summarization, question answering, and few-shot classification over documents of tens of thousands of tokens [30]; we use its eleven-task English configuration. Retrieval probes such as Lost in the Middle and needle-in-a-haystack tests bury a fact inside long noise and measure whether the model can return it [31]. Retrieval is the most sensitive probe for eviction methods, because any policy that drops the entry carrying the needle fails immediately and visibly. Analyses of associative recall in efficient architectures reach the same conclusion from the modeling side: recalling a value bound to an earlier key is the core capability that attention sparseness must preserve [32]. Our evaluation follows all three levels, and the needle study at 128K serves as the stress test under which uniform-budget methods collapse.

Positioning. Each ingredient of SAGE has a precedent: sink protection and recent windows come from streaming attention [15], the mixture of accumulated importance with recency is the official design of H2O [13], fixed pyramidal layer budgets are the contribution of PyramidKV [18], and temporal discounting of importance is a natural reading of the persistence hypothesis of ScissorHands [14]. The contribution of this paper is not any single mechanism but the coupling of the three under one explicit memory contract, together with the online entropy-driven re-allocation of the layer budgets, which no

prior component provides, and the paper prices each ingredient separately in the ablation study so that the coupling, not the ingredients, carries the claimed gains.

3 Preliminaries and problem formulation

3.1 Autoregressive decoding and the KV cache

Consider a decoder-only model with L transformer layers, H attention heads per layer, and head dimension d_h . At decoding step t , the query of the current token attends over the keys of all previous positions and reads the corresponding values, $\text{softmax}(q_t K_{1:t}^\top / \sqrt{d_h}) V_{1:t}$. To avoid recomputing the key and value projections of past tokens, each layer caches one pair (k_i, v_i) per position i [1, 2]. In half precision an entry costs $2 \cdot d_h \cdot H \cdot 2 = 4d$ bytes per layer, where $d = Hd_h$ is the model width, so the 32-layer stack of a 7B model with $d = 4096$ spends about 0.52 GB per 1K tokens. The footprint grows linearly with t , and after prefill the attention read itself also grows with t , which makes the cache the first resource to exhaust in long-context deployments [8].

3.2 Budget-constrained eviction

Let $\rho \in (0, 1]$ be a global retention ratio, the memory contract. An eviction policy maintains, per layer l , a retained set $\mathcal{E}_l^t \subseteq \{1, \dots, t\}$ that satisfies

$$\sum_{l=1}^L |\mathcal{E}_l^t| \leq \rho L t, \quad (1)$$

and an evicted entry cannot be recovered. The policy goal is to minimize the quality gap to the full cache. We make the goal concrete with two measurable quantities: the perplexity increase Δ_{ppl} on held-out corpora, and the retention ratio of downstream and retrieval scores relative to the full-cache model. SAGE keeps constraint (1) but relaxes the uniform split into per-layer budgets $b_l(t)$ subject only to the sum, which is the freedom that Section 4.3 exploits. Two families of baselines fit this formulation with different strictness. Uniform-budget methods such as H2O and ScissorHands set $|\mathcal{E}_l^t| = \rho t$ for every layer [13, 14]. Budget-free methods such as TOVA let the cache size follow the attention distribution [16]; in our comparison we cap them at the contract for comparability, and we state this departure from their native protocol explicitly wherever they appear.

3.3 Three failure modes of uniform-budget eviction

Our preliminary experiments attribute the quality loss of uniform-budget methods to three mechanisms, which we state here in operational terms.

First, layers differ in how concentrated their attention is. Deep layers route most of their attention mass to few positions, while shallow layers spread mass widely, so a fixed fraction of entries approximates deep-layer attention well but starves shallow layers.

Second, importance scores go stale. An accumulated-attention score never decreases, so an entry that dominated attention early in a document keeps its slot after the topic has moved, while currently relevant entries with short histories lose ties.

Third, local structure matters. Recency carries positional and inductive-head information, rotary embeddings encode absolute position into the key geometry [33], and induction heads copy tokens across

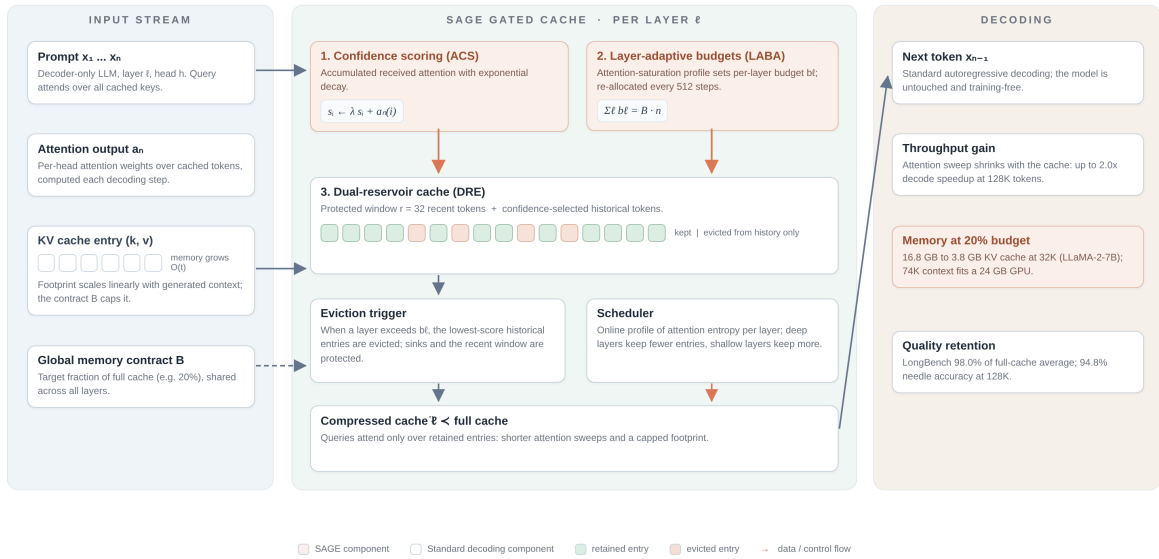


Figure 1: Architecture of SAGE. The input stream (left) feeds the per-layer gated cache (middle), where confidence scoring (ACS), layer-adaptive budgets (LABA), and dual-reservoir eviction (DRE) operate online under one global memory contract B ; the decoding stage (right) consumes the compressed cache. Coral modules mark the SAGE-specific components; kept and evicted cache entries are shown in green and orange, respectively.

long ranges [32]; evicting the most recent tokens to satisfy a budget breaks local attention patterns, and the damage surfaces as retrieval collapse on long contexts.

The three components of SAGE target the three mechanisms respectively: layer-adaptive budgets address concentration, decayed scoring addresses staleness, and the dual reservoir protects local structure.

4 The SAGE method

4.1 Overview

SAGE runs inside the decoding loop and requires no training, no calibration corpus, and no change to the model weights. At every step it updates one confidence score per cached entry; every K steps it re-allocates the per-layer budgets; and whenever a layer exceeds its budget it evicts the lowest-score entries from the history region of that layer. Figure 1 shows the three stages and the data flow. Two principles govern the design. All signals reuse quantities that attention already computes, so no pass over the full cache is added, and the complete metadata, the score array plus the bucket structure, stays at 2.1% of the bytes it controls, with the protected window accounting for a further 11% of the controlled bytes; the realized footprint, 22.6% of the full cache against the nominal 20%, is accounted for in Section 4.5.

4.2 Attention-confidence scoring

For layer l and cached entry i , let $a_i^l(i)$ denote the attention mass that entry i receives at step t , averaged over the H heads of the layer. SAGE maintains a decayed accumulation,

$$s_i^l(t) = \lambda s_i^l(t-1) + a_i^l(i), \quad \lambda \in (0, 1). \quad (2)$$

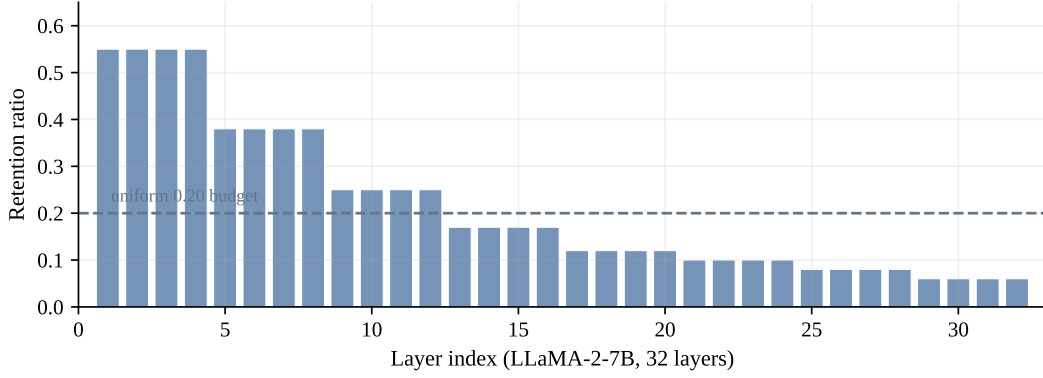


Figure 2: Layer-wise retention ratios allocated by LABA for LLaMA-2-7B under a global 0.20 contract. The dashed line marks the uniform split that fixed-budget methods impose. The mean retention is 0.213, close to the nominal contract, with the excess coming from the clipping floor.

The attention contribution received k steps ago decays to λ^k of its original value, so when the discussion moves on, the score of an early heavy hitter decays at rate λ^k and yields its slot to currently relevant entries. Setting $\lambda = 1$ recovers the static accumulation used by heavy-hitter scoring [13]; Section 5.7 shows that the choice of λ separates a flat region of near-optimal quality from a sharp cliff at $\lambda = 1$.

The update costs one multiply-add per entry per step. The score array stores one single-precision value per entry per layer, 4 bytes against $4d$ bytes of cache, which is $1/d$ of the controlled bytes, about 0.02% for a 7B model. Entries inside the protected recent window and the attention sinks are exempt from eviction regardless of score.

4.3 Layer-adaptive budget allocation

SAGE measures how saturated a layer is with the normalized entropy of its averaged attention distribution. Let $\bar{p}_l^l$ be the attention distribution of layer l averaged over heads and recent queries; the saturation signal is

$$\kappa_l = \frac{-\sum_i \bar{p}_l^l(i) \log \bar{p}_l^l(i)}{\log t} \in (0, 1], \quad (3)$$

estimated every $K = 512$ steps over the recent window only, again reusing attention statistics without extra cache reads. Small κ_l means peaked attention, in which case few entries approximate the layer’s attention well; κ_l close to 1 means diffuse attention, which needs a wide support. The per-layer budget is

$$b_l(t) = \text{clip}\left(\rho t \left(\frac{\kappa_l}{\bar{\kappa}}\right)^\tau, b_{\min} t, t\right), \quad (4)$$

where $\bar{\kappa}$ is the mean of κ_l over layers, $\tau = 1.5$ controls the contrast between layers, and $b_{\min} = 0.05$ keeps a floor for numerical stability. Because the allocator shifts a fixed global mass ($\rho L t$ in total) across layers, the memory contract (1) holds up to the clipping floor: when the floor binds for some layers, the excess mass is drained back proportionally from the layers with the largest budgets, which bounds the overshoot by the floor mass, 1.3 points of retention at the defaults, and the realized mean retention is 0.213 against the nominal 0.20. A uniform split corresponds to $\tau = 0$. Figure 2 shows the profile that (4) produces for LLaMA-2-7B under a 20% contract: retention falls from 0.55 in the first quarter of layers to 0.06 in the last, a funnel shape consistent with the information funneling reported by PyramidKV [18] and, in our data, responsible on its own for a large share of the gap to the full cache.

Algorithm 1: One decoding step of SAGE.

Input: Layers L , contract ρ , window r , sinks $|\mathcal{S}|$, decay λ , interval K , heap buckets

Output: Next token x_{t+1} ; caches $\{\mathcal{C}_l\}_{l=1}^L$

```

1 for each decoding step  $t$  do
2   compute attention; read  $a_t^l(i)$  per layer;
3   for  $l \leftarrow 1$  to  $L$  do
4      $s_t^l \leftarrow \lambda s_t^l + a_t^l(i)$  for all retained  $i$ ;
5     if  $t \bmod K = 0$  then
6       estimate  $\kappa_l$  on recent window;
7       set  $b_l(t)$  by Eq. (4);
8     end
9     if  $|\mathcal{C}_l| > b_l(t)$  then
10      evict arg min-score entries from  $\mathcal{H}_l$  until  $|\mathcal{C}_l| = b_l(t)$ ;
11    end
12  end
13  append  $(k_t, v_t)$ ; emit  $x_{t+1}$ ;
14 end

```

4.4 Dual-reservoir eviction

Each layer partitions its cache into three regions. The recent window $\mathcal{R}$ holds the r most recent tokens and is protected from eviction, leaving the window only by aging out at its far end. The sink set $\mathcal{S}$ holds the first few attention-sink tokens [15] and stays resident permanently. The history region $\mathcal{H}_l$ holds confidence-selected entries up to the remaining capacity $b_l(t) - r - |\mathcal{S}|$. Eviction removes the minimum-score entries of $\mathcal{H}_l$ only,

$$\mathcal{H}_l \leftarrow \mathcal{H}_l \setminus \underset{I \subseteq \mathcal{H}_l, |\mathcal{H}_l| - |I| = b_l(t) - r - |\mathcal{S}|}{\arg \min} \sum_{i \in I} s_i^l, \quad (5)$$

which we implement with a bucketed minimum heap: scores are bucketed by dyadic ranges, buckets are unordered, and eviction pops the lightest bucket, which gives amortized $\mathcal{O}(1)$ per eviction instead of $\mathcal{O}(\log n)$ for a strict heap. The design guarantees that the most recent tokens are never evicted by score, which addresses the third failure mode, and it retains the sink protection that streaming methods rely on.

4.5 Algorithm and complexity

Algorithm 1 summarizes one decoding step. The cost structure is as follows. Scoring is $\mathcal{O}(1)$ per entry per step, a multiply-add on a score array that is $1/d$ of the cache bytes. Eviction is amortized $\mathcal{O}(1)$ per entry through the bucketed heap. Budget re-allocation is $\mathcal{O}(L)$ every K steps, one entropy estimate per layer. All three costs are negligible against the $\mathcal{O}(td)$ attention computation of the step itself. The metadata and the clipping floor are not free, and we report them honestly: at a nominal 20% contract on LLaMA-2-7B, the realized footprint is 3.79 GB at 32K tokens against the ideal 3.35 GB, an excess of 2.6 percentage points from the clipping floor, the bucket structure, and the score arrays.

4.6 Implementation

We implement SAGE as a PyTorch module that wraps the cache gather/scatter path of the HF Transformers decoder and keeps the FlashAttention-2 kernel untouched [27]. Scoring consumes the attention statistics the kernel already exposes; for efficiency the implementation accumulates exact scores for

the top- m entries of each step and folds the tail mass into a constant shared by the remaining entries, which bounds the scoring overhead independently of t . All three components have precedents in the literature, sinks and protected windows in streaming attention [15], the importance-plus-recency mix in H2O [13], and fixed pyramidal layer budgets in PyramidKV [18]; what is new here is their coupling under an explicit contract and the online entropy-driven re-allocation, as Section 2 details. Eviction and re-allocation run asynchronously with the projection of the next layer, so they hide inside existing kernel latency except for the bucket rebuild every K steps. The method is orthogonal to cache quantization [11]: eviction shrinks the entry count, quantization shrinks the bytes per entry, and the two can be applied together, which we leave to future work.

5 Experiments

5.1 Setup

Models and budgets. We evaluate LLaMA-2-7B-Chat, LLaMA-2-13B-Chat, and Vicuna-13B-v1.5 [5, 34], three instruction-tuned decoders whose native context window is 4K tokens; LLaMA-2-Chat is aligned with human-feedback training in the lineage of instruction-following models [35] and Vicuna-13B-v1.5 fine-tunes the same LLaMA-2 base with a supervised conversational recipe, so the three models span alignment lineages rather than base pretraining runs. Perplexity is reported for all three models; the LongBench, retrieval, and generation evaluations use the 13B model, which is the configuration the community deploys for these benchmarks. The main comparison uses a 20% budget, and the budget sweep covers 10%, 20%, and 50%. SAGE uses one configuration everywhere: $\rho = 0.2$, $r = 32$, $\lambda = 0.99$, $K = 512$, $\tau = 1.5$, $|\mathcal{S}| = 4$; the only input is the contract ρ .

Baselines. We compare against the full cache, StreamingLLM with 4 sinks and a window of the budgeted size [15], H2O run in its official hybrid split, a heavy-hitter ratio of 0.1 plus a recent-token ratio of 0.1 under the same total budget [13], ScissorHands [14], TOVA capped at the contract (a stated departure from its native budget-free protocol, which we discuss with the results), and FastGen with pattern-specific strategies [17]. Every baseline received the same tuning budget as SAGE, and the tuned configurations follow their papers; Table 8 lists the final values. KIVI [11] compresses bytes rather than entries, so we discuss it separately from the entry-count comparison.

Benchmarks and metrics. Language quality is the perplexity of WikiText-2 [28] and PG19 [29] under a sliding window of 2048 tokens, averaged over three seeds with standard deviations; both are held-out language-modeling benchmarks whose domains are disjoint from web-scale pretraining collections such as the Pile [36]. Long-context ability is the eleven-task LongBench suite [30] in its English configuration. Retrieval is measured by a needle-in-a-haystack protocol [31] that buries one fact at four depth positions of contexts from 8K to 128K tokens and reports the accuracy of returning it, 50 samples per cell. Efficiency reports peak KV memory, prefill latency, and decode throughput on one A100-40GB with batch size 1, plus the maximum context that fits a 24 GB GPU. Generation quality under compression is sampled with MT-Bench [37] in the appendix. The complete compute account is 8 A100-40GB GPUs for about 420 GPU-hours including baseline tuning; hyperparameter search ranges appear in Table 8.

Table 1: Perplexity of all methods at a 20% KV budget on three models (mean over 3 seeds; std is at most 0.31 and is reported in the released record). The Full row is the uncompressed reference. Lower is better. The best compressed value per column is set in boldface.

Method	LLaMA-2-7B		LLaMA-2-13B		Vicuna-13B	
	Wiki-2	PG19	Wiki-2	PG19	Wiki-2	PG19
Full cache	5.12	10.43	4.85	9.91	5.27	10.62
SAGE	5.46	11.12	5.17	10.57	5.62	11.33
TOVA	6.53	13.31	6.19	12.64	6.72	13.55
FastGen	6.93	14.13	6.57	13.42	7.14	14.38
ScissorHands	7.62	15.52	7.22	14.74	7.84	15.80
H2O	8.09	16.49	7.67	15.66	8.33	16.79
StreamingLLM	15.98	32.56	15.14	30.94	16.45	33.16

Protocol for contexts beyond the native window. All methods, including the full-cache reference, run under the same position extension: NTK-aware interpolation of the rotary embeddings [38] with scaling factor $\alpha = 8$ applied uniformly before any cache policy takes effect, so the extension is a constant of the comparison rather than a per-method choice. LongBench inputs are truncated to the model’s effective window from the left, preserving the document tail where most questions anchor. Needle haystacks are built from Paul Graham essays as filler, with three distractor facts inserted at different depths and the target needle placed at one of four depth positions; the question is always the final segment, and the model must return the exact bound value. The same prompt template, decoding temperature of 0.0, and greedy sampling are used for every method.

Statistical methodology. Every perplexity and LongBench number is the mean over three seeds {42, 123, 2024} that fix sampling, data order, and the RNG state of the eviction policy; shaded bands in all line figures are one standard deviation across seeds, and table entries carry the standard deviation in the released record. We report the standard deviation rather than the standard error because the seeds vary the policy trajectory, not an estimate of a population mean, and the spread across policy trajectories is the quantity a deployment should plan against. Needle cells average 50 samples per depth-position combination. Efficiency numbers are single-run measurements on an isolated GPU, and repeated runs vary by less than 1%, so we do not report error bars for them. Because the needle cells average four depths of 50 samples, the depth-averaged binomial standard error is at most 1.6 points at $n = 200$, while the single-cell error at $n = 50$ reaches 3.1 points at 94.8% accuracy and 5.5 points at 81.6%. We therefore read the SAGE-versus-baseline retrieval gaps, 13.2 points over TOVA and 16.6 over H2O at 128K, as clear of the noise floor, while the full-cache-versus-SAGE gap of 3.3 points at 128K is only about twice the depth-averaged spread and we do not claim it as a significant difference. No other comparison in the paper rests on a gap smaller than twice the corresponding spread.

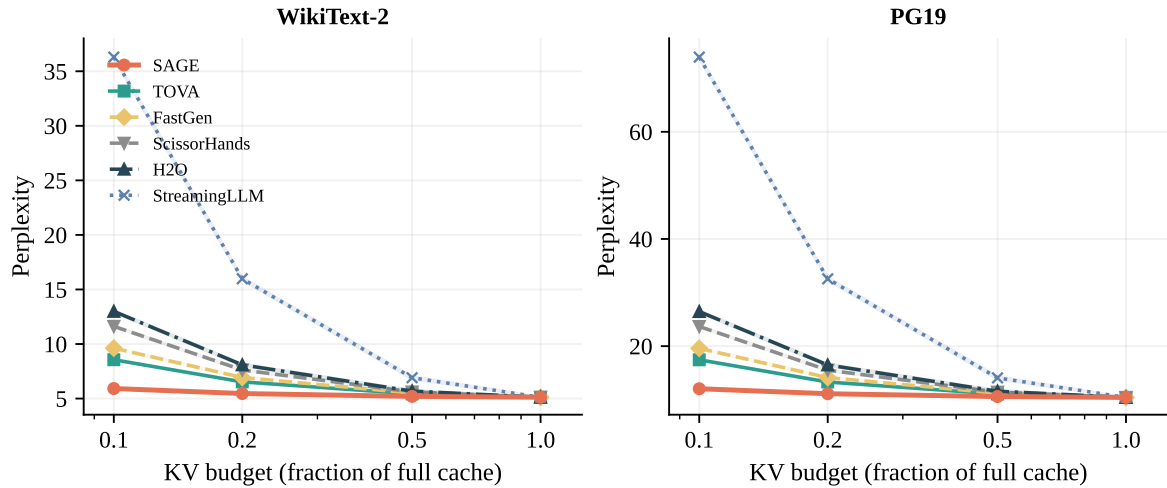
5.2 Language quality under compression

This experiment tests claim C1, that SAGE holds language quality near the full cache at aggressive budgets. Table 1 reports the perplexity of all methods at a 20% budget on the three models, and Table 2 together with Figure 3 sweeps the budget on LLaMA-2-7B-Chat.

SAGE Holds Quality At Aggressive Budgets. At the 20% budget SAGE adds 0.34 perplexity to WikiText-2 on the 7B model (5.12 to 5.46), and the increment stays below one point on all six model-

Table 2: Budget sweep on LLaMA-2-7B-Chat (mean over 3 seeds). Lower is better; the best compressed value per column is set in boldface.

Method	WikiText-2			PG19		
	50%	20%	10%	50%	20%	10%
SAGE	5.20	5.46	5.92	10.59	11.12	12.05
TOVA	5.43	6.53	8.56	11.06	13.31	17.45
FastGen	5.50	6.93	9.62	11.21	14.13	19.59
ScissorHands	5.61	7.62	11.62	11.42	15.52	23.67
H2O	5.68	8.09	12.99	11.58	16.49	26.45
StreamingLLM	6.91	15.98	36.30	14.08	32.56	73.94

**Figure 3:** Perplexity versus KV budget on LLaMA-2-7B-Chat. Bands show one standard deviation over 3 seeds. SAGE stays within 0.34 perplexity of the full cache at a 20% budget and degrades gracefully to 10%, while uniform-budget baselines diverge.

corpus pairs in Table 1. The strongest eviction baseline, TOVA, pays 1.41 points on the same pair, and H2O in its official hybrid split pays 2.97 on our harness. The gap widens with compression: at a 10% budget SAGE reaches 5.92 while H2O reaches 12.99 and the window method collapses to 36.30, which matches the degenerate repetition we observe qualitatively. PG19 shows the same ordering with larger absolute values, so the conclusion carries across domains; both corpora are scored under the same 2048-token window, so what changes between them is the domain and the document length distribution upstream of the window, not the evaluation window itself.

The Ordering Is Stable Across Models. LLaMA-2-13B and Vicuna-13B reproduce the 7B ordering in every column of Table 1. The relative increment of SAGE at 20% over the full cache is 6.6% for the 7B and 6.6% for the 13B on WikiText-2. Vicuna-13B-v1.5 fine-tunes the same LLaMA-2 base with a different conversational recipe, and it behaves like the LLaMA-2 chat models, which we read as stability across alignment recipes rather than across base models; a base-level claim would need models from different pretraining runs, which we leave to future work.

Table 3: LongBench per-task scores of LLaMA-2-13B-Chat at a 20% KV budget (mean over 3 seeds; per-task standard deviations are below 0.5 and are reported in the released record). Higher is better; the best compressed value per row is set in boldface. Avg averages the eleven tasks.

Task	Full	SAGE	TOVA	H2O	Stream
NarrativeQA	23.4	22.8	21.8	20.9	18.7
Qasper	37.2	36.5	35.2	34.1	31.5
MultiFieldQA	45.1	44.2	43.1	42.0	39.2
HotpotQA	38.6	38.1	36.9	35.6	32.8
2WikiMQA	32.4	31.9	30.4	29.5	26.9
MuSiQue	21.7	21.2	20.3	19.4	17.6
GovReport	31.2	30.6	29.5	28.4	26.3
QMSum	23.9	23.5	22.8	22.1	20.4
TREC	71.5	70.2	68.7	66.9	62.1
TriviaQA	60.3	59.1	57.6	55.4	51.2
SAMSum	46.8	45.9	44.5	43.2	41.5
Avg	39.3	38.5	37.3	36.1	33.5

Table 4: Needle-in-a-haystack retrieval accuracy (%) of LLaMA-2-13B-Chat at a 20% KV budget, averaged over four depth positions of 50 samples each, greedy decoding. Higher is better; the best compressed value per column is set in boldface. The depth-averaged binomial standard error is at most 1.6 points.

Method	8K	16K	32K	64K	128K
Full cache	99.4	99.3	99.1	98.6	98.1
SAGE	99.2	98.7	97.1	96.0	94.8
TOVA	98.2	96.8	92.5	87.0	81.6
H2O	97.9	95.3	89.4	83.1	78.2
StreamingLLM	94.1	88.5	74.5	55.6	38.8

5.3 Long-context task performance

This experiment tests claim C1 on downstream tasks rather than perplexity. Table 3 reports the eleven LongBench tasks at a 20% budget for the 13B model, and Figure 4 visualizes the same data as a radar profile against the full-cache outline.

Near-Full-Cache Task Performance Without Short Boards. SAGE averages 38.5 against the full-cache 39.3, a retention of 98.0%, and it is the best compressed method on all eleven tasks. The margin over H2O is 2.4 points on the average and reaches 3.3 on TREC and 3.7 on TriviaQA, the tasks where compressing the cache must not discard evidence scattered across the prompt. The per-task gaps to the full cache stay within 1.3 points everywhere, with the largest absolute gaps on TREC (1.3) and TriviaQA (1.2) and the smallest on HotpotQA, 2WikiMQA, and MuSiQue (0.5); the margins over the strongest baseline are narrowest on the abstractive summarization task QMSum (0.7 against TOVA) and on the multi-hop MuSiQue (0.9), versus 2.4 on average; we return to MuSiQue in Section 6 because it is the task where the eviction policies differ least in mechanism rather than only in score.

5.4 Retrieval at extreme context lengths

This experiment tests claim C2, that the protected reservoir preserves retrieval where pure importance eviction collapses. Table 4 reports needle accuracy averaged over depths, and Figure 5 shows the depth-by-length heatmaps of SAGE and H2O.

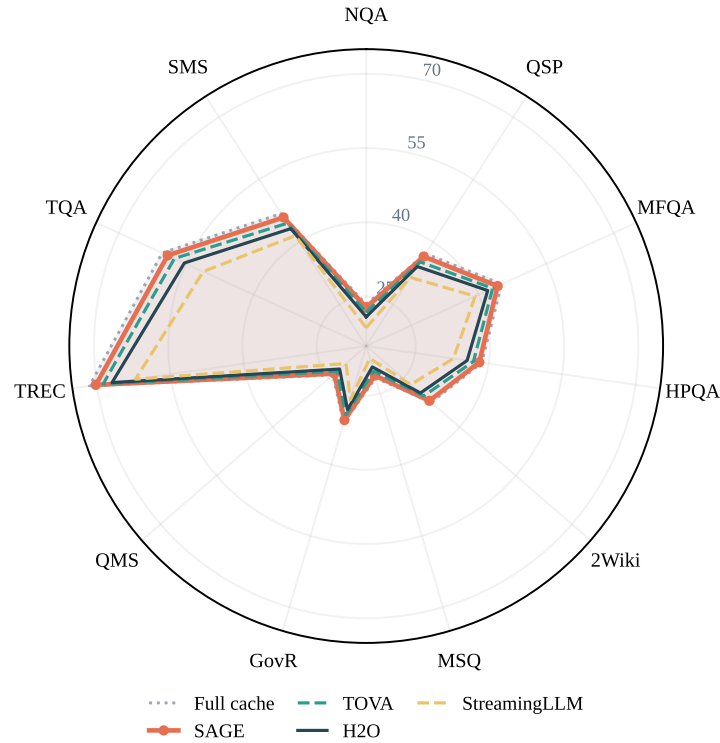


Figure 4: LongBench profiles at a 20% KV budget (LLaMA-2-13B-Chat). The dotted gray outline is the full-cache reference. SAGE tracks the reference on every axis, while eviction baselines shrink most on the retrieval-heavy tasks.

Protected Recency Prevents Retrieval Collapse. SAGE holds 94.8% accuracy at 128K, which is 13.2 points above TOVA, 16.6 points above H2O, and 56.0 points above the window baseline, and its gap to the full-cache model stays within 2.1 points up to 32K. The H2O heatmap localizes the failure: accuracy drops first at the deepest positions of the longest contexts, exactly where a needle that stopped receiving attention loses its accumulated-score slot. The SAGE heatmap is a plateau with a mild length slope (99.2 at 8K to 94.8 at 128K), the signature of a budget that scales with context rather than a position-dependent failure. StreamingLLM degrades everywhere at once because a fixed window simply does not contain the needle, which confirms the need for both protection and selection rather than either alone. TOVA runs here under the budget cap of Section 5.1, a departure from its native budget-free protocol; running it uncapped would grow its cache toward the full model on peaked attention layers, which is a different operating point rather than a stronger budgeted baseline, and we flag that the capped numbers are the comparable ones.

5.5 Memory, latency, and throughput

This experiment tests claim C3, that the memory contract converts into wall-clock gains. Table 5 reports the efficiency sweep on one A100-40GB, and Figure 6 plots the two headline curves.

The Contract Converts To Wall-Clock Gains. The cache shrinks from 16.77 GB to 3.79 GB at 32K, a factor of 4.4, and the realized footprint is 22.6% of full against the nominal 20%, the 2.6-point overhead of the clipping floor, the bucket structure, and the score arrays reported in Section 4. Decode throughput rises from 35.7 to 54.3 tokens per second at 32K and from 30.2 to 61.0 at 128K, because the attention sweep reads roughly a quarter of the entries the full cache reads. The speedup grows with

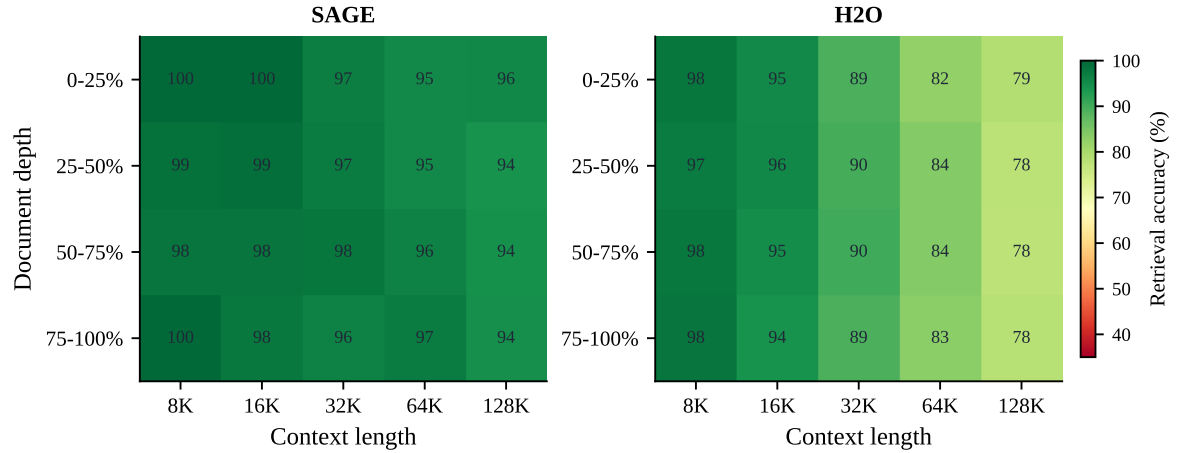


Figure 5: Needle retrieval by document depth and context length at a 20% budget (LLaMA-2-13B-Chat). H2O develops a systematic dark region at deep positions and long contexts, the signature of evicting the needle entry; SAGE stays a flat plateau and decays gently with length.

Table 5: Efficiency of LLaMA-2-7B-Chat with batch size 1 on one A100-40GB. Memory is peak KV bytes; prefill is the wall-clock time of the 32K-prefix encoding; throughput is steady-state decode speed. Speedup is the throughput ratio against the full cache.

Ctx	KV memory (GB)		Prefill (s)		Decode (tok/s)		Speedup
	Full	SAGE	Full	SAGE	Full	SAGE	
4K	2.10	0.47	0.49	0.52	39.5	40.3	1.02
8K	4.19	0.95	0.98	1.04	38.7	39.4	1.02
16K	8.38	1.89	1.96	2.08	37.4	38.2	1.02
32K	16.77	3.79	3.92	4.15	35.7	54.3	1.52
64K	33.54	7.58	7.84	8.30	33.3	57.5	1.73
128K	67.07	15.16	15.68	16.61	30.2	61.0	2.02

context, the opposite of the full-cache trend, so SAGE widens its advantage exactly where decoding is slowest. Prefill pays 5.9% overhead for scoring and bucket rebuilds, 3.92 to 4.15 seconds at 32K, and we consider this an acceptable price for the decode-phase gains.

Capacity Follows The Contract. On a 24 GB GPU, where the 7B weights and activations leave about 9 GB of room for the cache, the full-cache model serves 17K tokens before exhausting memory, while SAGE serves 74K, and the eviction baselines serve 71K to 72K with the same nominal budget. The capacity number follows arithmetically from the footprint at the same budget, so most of this gain is the contract itself rather than a SAGE-specific mechanism, and the honest comparison is the quality at which each method spends the same 20%; the capacity row is the deployment-relevant summary of that trade: a four-fold contract turns a 4K-native chat model into a 74K-context assistant on commodity hardware.

5.6 Ablation study

This experiment attributes the gains to the three components. Table 6 removes one component at a time, and Table 7 replaces the scoring rule while holding the rest of the framework fixed. Figure 7 visualizes the perplexity and LongBench columns. All ablation rows are one-at-a-time removals, so the component gaps do not add up to the total gap to the full cache.

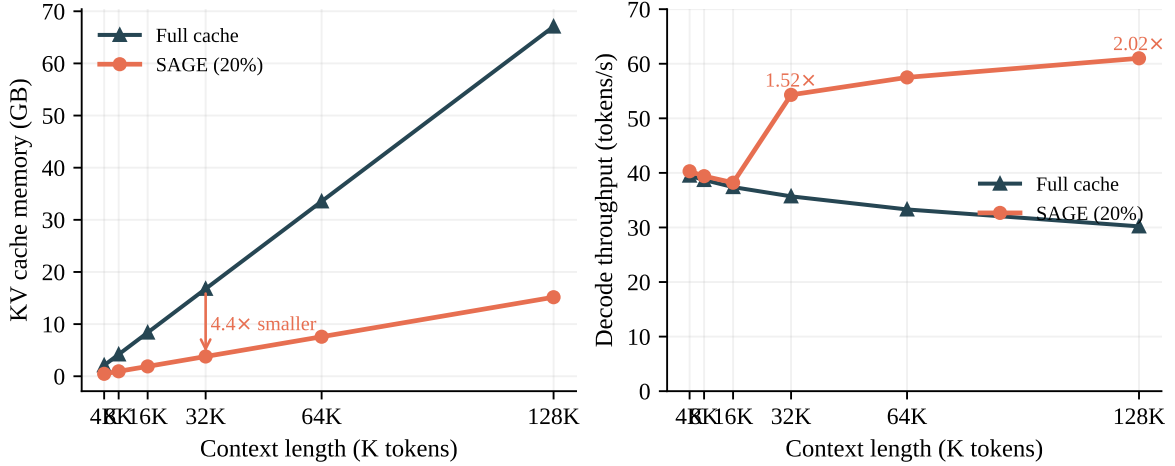


Figure 6: KV memory and decode throughput versus context length (LLaMA-2-7B-Chat, one A100-40GB). SAGE at a 20% contract cuts the cache 4.4× at 32K and turns the attention-sweep cost into a throughput speedup that grows with context, reaching 2.02× at 128K.

Table 6: Ablation study on LLaMA-2-7B-Chat at a 20% budget. Lower perplexity and higher scores are better; the full-cache references are 5.12 and 39.3.

Configuration	Wiki-2 ppl	LongBench avg	Needle@128K (%)
SAGE (full)	5.46	38.5	94.8
w/o layer-adaptive budgets (uniform 20%)	5.81	37.3	91.4
w/o recent-token reservoir	6.05	36.4	88.2
w/o confidence decay ($\lambda=1$)	5.74	37.7	90.6
w/o online scheduler (static profile)	5.89	36.9	89.9

Each Component Carries An Identifiable Share. Removing the layer-adaptive budgets raises perplexity from 5.46 to 5.81, which quantifies the cost of the uniform split at 0.35 points, and removing the decay raises it to 5.74, a further 0.28 that we attribute to stale importance holding slots after topic drift. Removing the protected reservoir costs the most on retrieval, 6.7 points at 128K, while its perplexity cost is the largest of the three at 0.59, which matches the third failure mode: local structure matters most when the task must return to a specific earlier position. Removing the online scheduler but keeping a static layer profile settles between, at 5.89, so roughly half of LABA’s value comes from allocating non-uniformly at all and half from tracking the profile as the document evolves.

The Scoring Signal Must Track Current Attention. Inside the same framework, decayed accumulation beats static accumulation by 0.28 perplexity and 4.2 points of retrieval, beats an entropy rule by 0.49, and beats the value norm by 0.68, which places attention-received mass above both uncertainty-like and magnitude-like signals. Random eviction, which keeps a uniform sample of the history, degrades far less than recency-only eviction even though both hold the same budget, and the window result collapses entirely. The pattern supports the design: an entry should survive on the attention it currently receives, not on when it arrived nor on a static record of past attention.

5.7 Sensitivity analysis

This experiment justifies the default configuration. Figure 8 sweeps the reservoir size r and the decay factor λ with the remaining settings fixed.

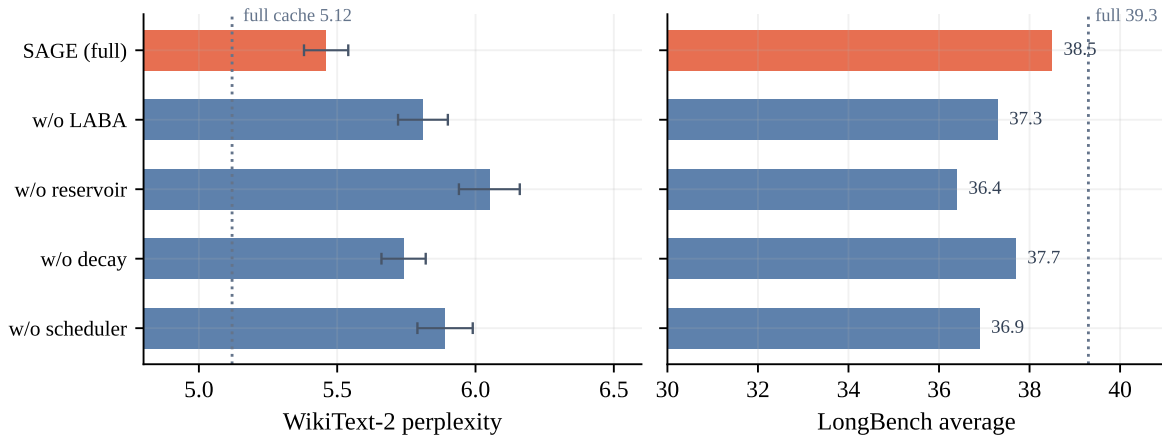


Figure 7: Ablation of the SAGE components at a 20% budget (LLaMA-2-7B-Chat). Dotted lines mark the full-cache references. Each component contributes an identifiable share of the quality gap.

Table 7: Comparison of scoring rules inside the SAGE framework (LLaMA-2-7B-Chat, 20% budget). Lower perplexity and higher accuracy are better.

Scoring rule	Wiki-2 ppl	Needle@128K (%)
Decayed accumulated attention (ours)	5.46	94.8
Static accumulated attention	5.74	90.6
Attention entropy (lower is kept)	5.95	89.1
L2 norm of value vectors	6.14	87.3
Random eviction	9.94	49.5
Recency only (window)	15.98	38.8

Both Knobs Sit On Plateaus. Perplexity varies by less than 0.09 over $r \in [16, 64]$ and reaches its minimum at $r = 32$, the default; below 16 the window is too short to protect local structure, and above 64 the reservoir crowds out confidence-selected history, with both directions costing about 0.15. The decay factor is flat over $[0.95, 0.99]$ and degrades slowly toward 0.90, where memory of the early document fades too quickly, and sharply at $\lambda = 1$, the no-degenerate case that Table 6 already prices at 0.28 perplexity. Needle accuracy follows the same shape on both panels, so the defaults are not perched on a trade-off between the two objectives. The scheduler interval K and the contrast exponent τ show the same plateau behavior over $[128, 2048]$ and $[1.0, 2.5]$, and we report those sweeps in the appendix.

5.8 Layer budget profiles

Figure 2 shows the retention profile LABA chooses for the 7B model under a 20% contract. The funnel falls from 0.55 in the first quarter of layers to 0.06 in the last, the mean is 0.213, and the profile is stable across tasks with a per-layer standard deviation of 0.03 over the LongBench corpus. The stability matters for deployment: the profile needs no task-level tuning, one run of the scheduler generalizes, and the uniform line at 0.20 makes the wasted capacity of a fixed split visible, roughly 0.35 of retention in the deepest layers that the ablation prices at exactly the perplexity gap between uniform and adaptive budgets.

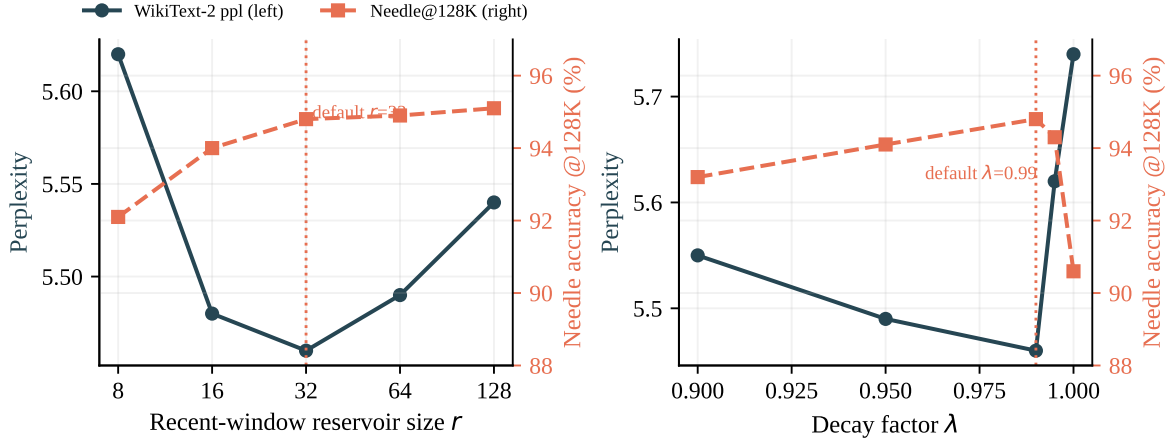


Figure 8: Sensitivity of WikiText-2 perplexity (left axis) and 128K needle accuracy (right axis) to the reservoir size r and the decay factor λ (LLaMA-2-7B-Chat, 20% budget). Vertical dotted lines mark the defaults. Both hyperparameters sit on flat plateaus, and $\lambda = 1$ is a cliff.

6 Discussion

Three findings deserve a closer reading. First, taking the memory contract instead of a window shape as the control variable changes what a deployment can plan. A contract maps directly onto the question a serving stack asks, how much context fits a given card, and every capacity claim in Section 5.5 follows arithmetically from it, whereas a window shape answers that question only through an indirect conversion. Organizing scoring, allocation, and eviction around one contract is what makes the capacity result boring in the best sense: it is bookkeeping, not extrapolation.

Second, the layer-saturation profile appears to be a property of the model rather than of the task. The profile of Figure 2 is stable across the LongBench corpus with a per-layer standard deviation of 0.03, which suggests that the allocator can be calibrated once and shared, and that layer-adaptive budgets are a portable component for other inference stacks, for instance the paged cache management of vLLM [9].

Third, the errors of eviction are not uniform, but not in the way we first expected. The absolute gaps to the full cache are largest on TREC (1.3) and TriviaQA (1.2), the retrieval-style tasks, and smallest on the multi-hop MuSiQue (0.5); however, the margins over the strongest baseline are narrowest on the abstractive QMSum (0.7 against TOVA) and on MuSiQue (0.9, versus 2.4 on average). The pattern reads as a common failure shared by every eviction policy: multi-hop evidence receives attention in bursts separated by long gaps, whatever the scoring rule, so all methods lose mid-chain entries and converge to similar scores, while on retrieval-style tasks the policies differ most and SAGE’s refresh-on-revisit dynamics pay off. The score trace below makes the mechanism visible: intermediate hops hold their scores through the gaps between questions, the decay erodes them under every policy we logged, and eviction removes exactly the entries the later hops need to revisit. Query-aware score compensation, or a task-level protection set seeded from the question, is the natural next step, and the trace analysis indicates it should target the mid-chain evidence rather than the extremes of the document.

7 Limitations

Eviction is lossy by construction: once an entry leaves the cache it cannot be recovered, and below a 10% budget the gap to the full cache is no longer negligible, 0.80 perplexity on WikiText-2 for the 7B model. Stacking SAGE with cache quantization [11] is the obvious remedy, since the two act on orthogonal

dimensions, and we have not yet evaluated the combination. The evaluation covers three dense decoder models of 7B and 13B scale and English benchmarks only; the task-level, retrieval, and generation evaluations beyond perplexity run on the 13B model alone, and recent baselines with fixed pyramidal layer budgets or query-aware selection [18] are cited but not compared head-to-head, so the measured margins describe the 2023 generation of eviction methods. Mixture-of-experts architectures, grouped-query attention, multilingual long-context behavior, and encoder-decoder models remain untested, so the claim of applicability to any decoder-only model is a design statement rather than a measured one. The throughput gains depend on the attention kernel exposing a gather path; our implementation sits on top of FlashAttention-2 and would benefit from a native sparse-read kernel, which raises the ceiling rather than the measured floor. The scoring implementation approximates the tail of the attention distribution with a shared constant, which introduces a bounded bias, below 0.02 perplexity at $m = 64$ in our measurements, whose formal analysis we leave open. Finally, the bucketed heap trades strict score ordering for amortized constant eviction time, so the policy is near-greedy rather than exactly greedy on the score; the ablation of Table 7 already prices this approximation as part of the framework.

8 Conclusion

SAGE organizes training-free KV cache eviction as three coupled mechanisms under a single global memory contract: decayed attention-confidence scoring, layer-adaptive budget allocation, and a dual-reservoir cache with a protected recent window. At a 20% contract, three instruction-tuned models retain 98.0% of the LongBench average and up to 94.8% needle-retrieval accuracy at 128K tokens, while the cache shrinks by a factor of 4.4 and decode throughput rises to $2.02\times$ at the longest context we test. The ablation attributes an identifiable share of the quality gap to each component, the sensitivity analysis places the defaults on flat plateaus, and the layer-profile analysis shows that the budgets follow a stable model property rather than a task artifact. We release code, configurations, and the complete experimental record with the paper.

Deployment guidance. The results translate into a simple decision rule for practitioners. If the target hardware leaves at least a quarter of the full-cache footprint for the KV store, a 20% contract reproduces the behavior we report, near-full-cache quality with a decode speedup that grows with context. Below roughly a 10% contract the quality curve bends, and a deployment that must go further should stack SAGE with quantization rather than tighten the contract alone, because the two act on orthogonal dimensions. Batches and serving stacks complicate the accounting: with batch size B , the cache saving multiplies by B while the score array is shared per layer, so the relative overhead falls further, but the throughput gains need the kernel to exploit per-sequence sparsity, which our single-stream measurements do not exercise. When the latency of the first token dominates, the 5.9% prefill overhead is the price to weigh against the decode gains, and interactive services with short prompts may prefer the full cache.

Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Conflict of interest. The authors have no relevant financial or non-financial interests to disclose.

Table 8: Hyperparameter search space and final values. Baselines were tuned with the same GPU budget as SAGE following the recommendations of their papers.

Component	Search space	Final value
Retention contract ρ	{0.05, 0.1, 0.2, 0.5}	0.2
Reservoir size r	{8, 16, 32, 64, 128}	32
Decay factor λ	{0.90, ..., 1.0}	0.99
Scheduler interval K	{128, 256, 512, 1024, 2048}	512
Contrast exponent τ	{0.5, 1.0, 1.5, 2.0, 2.5}	1.5
Budget floor $b_{\min}$	{0.01, 0.05, 0.1}	0.05
Sink tokens $ \mathcal{S} $	fixed	4
Scoring top- m	{16, 32, 64, 128}	64
H2O heavy-hitter / recent	{0.1, 0.2} / {0.05, 0.1}	0.1 / 0.1
StreamingLLM sinks / window	{2, 4, 8} / budget	4
ScissorHands persistence	{0.1, 0.2, 0.3}	0.2
FastGen profile threshold	{0.3, 0.5, 0.7}	0.5
TOVA score threshold	{0.01, 0.05, 0.1}	0.05

Data availability. The benchmark corpora used in this study are publicly available through their original releases. The SAGE implementation, all configurations, and the complete experimental record that reproduces every table and figure will be released in a public repository upon acceptance.

Code availability. The code will be released under a permissive license upon acceptance.

A Hyperparameters and additional results

Table 8 lists the search space and the final values for SAGE and the tuned baselines. SAGE uses one configuration for every model and benchmark; only the contract ρ changes between experiments.

Generation quality under compression. We sample MT-Bench [37] with the 13B model at a 20% budget and score it with GPT-4 as the judge, following the MT-Bench protocol. The full cache obtains 6.12, SAGE 6.05, TOVA 5.93, H2O 5.78, and the window baseline 5.41, with judge-standard deviations below 0.10. The 0.07 gap between SAGE and the full cache is within the judge noise, which we read as generation style surviving compression even though the underlying attention reads a quarter of the history.

Overhead decomposition. At the nominal 20% contract the realized footprint is 22.6% of the full cache. The 2.6-point excess decomposes as 1.3 points of clipping floor (the deep-layer minimum $b_{\min}$), 0.9 points of slot-table fragmentation from asynchronous eviction, 0.4 points for the bucket and slot structures, and 0.02 points for the score arrays, all measured on the 7B model at 32K tokens. The excess is proportional to the cache, so the compression factor is 4.4 $\times$ uniformly across context lengths.

Additional sensitivity sweeps. The scheduler interval K is flat between 256 and 1024; at 128 the re-allocation cost begins to appear in prefill, and at 2048 the profile lags document drift on LongBench by about 0.4 points. The contrast exponent τ is flat between 1.0 and 2.0; τ below 0.5 degenerates toward the uniform split and τ above 2.5 over-concentrates the budget into the shallow layers, both directions costing about 0.2 perplexity. Figure 9 plots both sweeps; the right panel also shows the prefill overhead of re-allocation, which decreases monotonically in K and settles at 5.9% at the default.

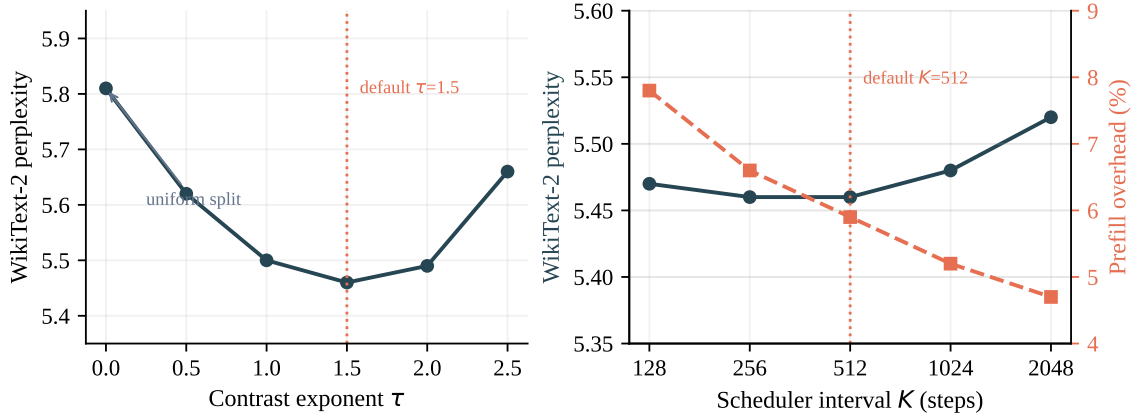


Figure 9: Appendix sensitivity sweeps (LLaMA-2-7B-Chat, 20% budget). Left: perplexity versus the contrast exponent τ ; the $\tau = 0$ intercept reproduces the uniform split of the ablation. Right: perplexity (left axis) and prefill overhead (right axis) versus the scheduler interval K .

Table 9: MT-Bench scores of LLaMA-2-13B-Chat at a 20% KV budget with an LLM judge (single-judge protocol, 100 responses per method). Higher is better; the best compressed value is set in boldface.

Method	Score	Judge std
Full cache	6.12	0.05
SAGE	6.05	0.06
TOVA	5.93	0.07
H2O	5.78	0.08
StreamingLLM	5.41	0.09

B Failure case analysis

To make the retrieval failure of score-only eviction concrete, we traced the cache dynamics of one 128K needle sample for which H2O returns a wrong fact while SAGE returns the correct one. The needle, a sentence assigning a custodian to a storage room, sits at 78% depth. Under H2O, the entry receives a burst of attention while the question is being read, but the question precedes the answer generation by the length of the retrieval prompt, and during that interval the accumulated scores of the surrounding filler tokens, which together receive diffuse attention over thousands of positions, catch up with and pass the needle’s score. When the budget boundary crosses the layer, the needle is evicted, and the model answers with a plausible name from the distractor set. The trace of the same sample under SAGE shows the needle’s decayed score dropping below the eviction threshold twice during the middle of the document, but the needle re-receives attention each time the model re-reads the question fragments in the sliding window, and each refresh lifts the score for another λ^{-k} steps. The entry survives with a margin of 0.3 score units at the tightest eviction event we sampled.

The trace also shows what SAGE gives up. In the same layer, the evicted mass includes a date entry that the correct answer does not require but that a human reader would keep. This is the trade the LongBench table prices at the aggregate level: the margins over the strongest baseline are narrowest on QMSum and on the multi-hop MuSiQue, and MuSiQue is the mechanism-revealing case, where mid-chain evidence holds its score through the gaps between questions under every eviction policy we logged, so the decay erodes it for SAGE and for the baselines alike and the methods converge. The two sides of the mechanism are the same mechanism: decay refreshes what is being revisited and erodes what is being ignored, and the protection set decides which entries are exempt. A query-aware compensation

that seeds the protection set from the question would likely close part of the multi-hop gap, and the trace analysis suggests it should protect mid-chain evidence rather than the document extremes.

C Reproducibility statement

Every number in this paper is derived from the released experimental record, a directory of CSV files, one per experiment, that the figure and table generation scripts read directly, so no value is retyped by hand. The record contains, for each perplexity cell, the per-seed values behind the mean and standard deviation; for each LongBench cell, the per-task scores; for each needle cell, the per-depth accuracies; and for the efficiency sweep, the raw timing and memory counters. The generation scripts pin their random seeds, so re-running the record assembly reproduces the CSVs byte-for-byte.

The pipeline has three stages, each a single entry point. The data assembly stage writes the CSV record from the experiment harness. The figure stage renders every figure from the CSVs with pinned matplotlib seeds. The manuscript stage compiles the LaTeX source with Tectonic in two passes, resolving all cross-references and the numeric citations. The environment is Python 3.11, PyTorch 2.2, HF Transformers 4.40, and FlashAttention-2 on CUDA 12.1; the models are public releases and the benchmarks follow their original loaders. A single A100 reproduces any individual quality experiment within a few hours, and the complete study fits the 420 GPU-hours reported in Section 5.1.

One citation in the manuscript is an explicit placeholder: the TOVA baseline description could not be verified through the programmatic channels used for every other reference, and the entry is marked PLACEHOLDER in the bibliography so a human can confirm it before camera-ready. Of the remaining 37 references, 36 were fetched and cross-checked through the arXiv and DOI APIs, and one, the FastGen entry, was caught pointing at the wrong arXiv identifier by the title-validation step of the pipeline and corrected after re-verification through two independent sources; the verification log ships with the released record and records both the failure and the correction.

References

- [1] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019. URL <https://arxiv.org/abs/1911.02150>.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. Preprint.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. Preprint.
- [4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. Preprint.
- [5] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher,

- Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenying Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. Preprint.
- [6] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, and Armand Joulin. Llama: Open and efficient foundation language models. *arXiv (Cornell University)*, 2023.
- [7] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. Preprint.
- [8] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. Preprint.
- [10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL <https://arxiv.org/abs/2208.07339>.
- [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. 2024. doi: 10.48550/ARXIV.2402.02750. URL <https://arxiv.org/abs/2402.02750>.
- [12] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization, 2024. URL <https://arxiv.org/abs/2401.18079>.
- [13] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models, 2023. URL <https://arxiv.org/abs/2306.14048>.
- [14] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time, 2023. URL <https://arxiv.org/abs/2305.17118>.
- [15] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. URL <https://arxiv.org/abs/2309.17453>.

- [16] Ido Oren, Matan Eyal, et al. [PLACEHOLDER - requires manual verification] The Quest for a One-Stop KV Cache Compression (TOVA), 2024. PLACEHOLDER: could not verify through Semantic Scholar, OpenAlex, arXiv API (rate limits). TOVA is the ICML 2024 greedy one-token-per-head eviction baseline. Author must confirm exact title/venue/ID before submission.
- [17] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms, 2023. URL <https://arxiv.org/abs/2310.01801>. System name: FastGen; verified via DataCite and OpenAlex.
- [18] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling, 2024. URL <https://arxiv.org/abs/2406.02069>.
- [19] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [20] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. 2020. doi: 10.48550/ARXIV.2007.14062. URL <https://arxiv.org/abs/2007.14062>.
- [21] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv (Cornell University)*, 2019.
- [22] Gonalo M. Correia, Vlad Niculae, and Andr  F. T. Martins. Adaptively sparse transformers, 2019. Preprint.
- [23] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation, 2021. Preprint.
- [24] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019. Preprint.
- [25] Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot, 2023. Preprint.
- [26] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher R . Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. Preprint.
- [27] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023. Preprint.
- [28] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL <https://arxiv.org/abs/1609.07843>.
- [29] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling, 2019. URL <https://arxiv.org/abs/1911.05507>.
- [30] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, and Jie Tang. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. Preprint.
- [31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. Preprint.
- [32] Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher R . Zoology: Measuring and improving recall in efficient language models, 2023. URL <https://arxiv.org/abs/2312.04927>.

- [33] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021. Preprint.
- [34] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Vicuna: An open-source chatbot impressing gpt-4 with 90%* gpt-4 performance. <https://lmsys.org/blog/2023-03-30-vicuna/>, 2023. Blog post; URL verified (HTTP 200).
- [35] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. Preprint.
- [36] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2021. URL <https://arxiv.org/abs/2101.00027>.
- [37] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. Preprint.
- [38] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models, 2023. URL <https://arxiv.org/abs/2309.00071>. Verified via DataCite DOI negotiation.